LAB | Automating AWS IAM User Creation with Python + boto3 Extra

Skills you will learn

Cloud Computing: Identity & Security

> ***Context*** *– Students have already automated IAM user creation with a Bash script. In this follow-up, students will* ***re-implement the whole workflow in Python****, gaining practice with virtual-envs,* `boto3`*, and Python scripting conventions.*

Objectives

By the end of the lab each student will be able to:

1. Create and activate a Python virtual environment for AWS automation.
2. Write a step-by-step Python script that
   - reads user details from a CSV file,
   - generates secure random passwords,
   - creates IAM users programmatically,
   - attaches the **AmazonS3ReadOnlyAccess** policy, and
   - stores credentials locally in a protected file.
3. Run, verify, and (optionally) clean up the created resources.

0. Prerequisites

Item	Check-list
AWS CLI configured	`aws configure list` shows valid keys/region.
IAM permissions	The student's IAM role **must** allow `iam:*User*`, `iam:AttachUserPolicy`, `iam:CreateLoginProfile`.

Ask AI Tutor

Item	Check-list
Python 3.9 +	`python3 --version` ⟹ **≥ 3.9** (for `secrets` & f-strings).
Local tools	`pip`, `virtualenv` (or the built-in `venv` module).
CSV file	Copy `employees.csv` into the working directory (same structure as in the Bash lab).

1. Project Setup

1.1 Create a clean workspace

```
mkdir iam-python-lab && cd iam-python-lab
python3 -m venv .venv          # create isolated env
source .venv/bin/activate      # Linux / macOS
# .venv\Scripts\activate.bat   # PowerShell / Windows
pip install --upgrade pip boto3 python-dotenv
```

Copy Explain

Why `python-dotenv`? – We will keep the AWS keys in the same profile the CLI already uses, **but** it is handy to demonstrate loading variables from a `.env` file for future projects.

1.2 Folder structure

```
iam-python-lab/
│
├─ create_iam_users.py      # main script (built step-by-step)
├─ employees.csv            # input
└─ credentials.csv          # output (auto-generated)
```

2. Script Creation – Step by Step

Open `create_iam_users.py` in your favourite editor and add each section **in order**, running the script between steps to see incremental progress.

2.1 Imports & AWS session

```
#!/usr/bin/env python3
import csv
import pathlib
import secrets
import string
import sys
from datetime import datetime

import boto3
```

```
from botocore.exceptions import ClientError
```

Create an AWS session that re-uses the CLI's default profile:

```
session = boto3.Session()  # uses credentials in ~/.aws/credentials
iam = session.client("iam")
```

Copy Explain

2.2 Helper: generate a strong password

```
def gen_password(length: int = 16) -> str:
    alphabet = string.ascii_letters + string.digits + string.punctuation
    while True:                                  # enforce at least one of each
        pwd = "".join(secrets.choice(alphabet) for _ in range(length))
        if (any(c.islower() for c in pwd) and
            any(c.isupper() for c in pwd) and
            any(c.isdigit() for c in pwd) and
            any(c in string.punctuation for c in pwd)):
            return pwd
```

Copy Explain

2.3 Core function: create a single IAM user

```
POLICY_ARN = "arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess"

def create_user(username: str, password: str) -> None:
    try:
        iam.create_user(UserName=username)
        print(f"✅ User {username} created")

        iam.create_login_profile(
            UserName=username,
            Password=password,
            PasswordResetRequired=True,
        )
        print(f"🔑 Login profile set")

        iam.attach_user_policy(UserName=username, PolicyArn=POLICY_ARN)
        print(f"📜 Policy attached\n")

    except ClientError as e:
        print(f"⚠️ Skipped {username}: {e.response['Error']['Message']}")
```

Copy Explain

2.4 Read the CSV and process each row

```
INPUT_CSV = pathlib.Path("employees.csv")
OUTPUT_CSV = pathlib.Path("credentials.csv")

def main() -> None:
    if not INPUT_CSV.exists():
```

```
        sys.exit("employees.csv not found - aborting.")

    first_run = not OUTPUT_CSV.exists()
    with open(INPUT_CSV, newline="") as infile, \
         open(OUTPUT_CSV, "a", newline="") as outfile:

        reader = csv.DictReader(infile)
        writer = csv.writer(outfile)

        if first_run:
            writer.writerow(["username", "password", "created_at"])

        for row in reader:
            username = f"{row['first_name'].lower()}.{row['last_name'].lower()}"
            password = gen_password()

            create_user(username, password)
            writer.writerow([username, password, datetime.utcnow().isoformat()])

if __name__ == "__main__":
    main()
```

2.5 Lock down the credentials file

After the first successful run:

```
chmod 600 credentials.csv
```

Copy Explain

*(Explain to students **why**—to prevent accidental exposure of plaintext passwords.)*

3. Running the Script

1. **Activate env & run**

```
source .venv/bin/activate
python create_iam_users.py
```

Copy Explain

2. **Verify in AWS Console or CLI**

```
aws iam list-users --query 'Users[*].UserName'
```

Copy Explain

3. **Inspect `credentials.csv`** – you should see usernames, passwords, and timestamps.

4. Optional Cleanup Script

Create `cleanup_iam_users.py`:

```python
#!/usr/bin/env python3
import csv
import boto3

iam = boto3.client("iam")
POLICY_ARN = "arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess"

with open("credentials.csv") as f:
    next(f)  # skip header
    for line in f:
        username = line.split(",")[0]

        try:
            iam.delete_login_profile(UserName=username)
            iam.detach_user_policy(UserName=username, PolicyArn=POLICY_ARN)
            iam.delete_user(UserName=username)
            print(f"🗑️  Deleted {username}")
        except iam.exceptions.NoSuchEntityException:
            print(f"⚠️  {username} already gone")
```

Copy Explain

Run with

```
python cleanup_iam_users.py
```

Copy Explain

5. Completion Checklist

- [ ] All IAM users from `employees.csv` appear in **IAM → Users**.
- [ ] `credentials.csv` exists, **permissions 600**, and lists username + password.
- [ ] Each user has the **AmazonS3ReadOnlyAccess** policy attached.
- [ ] *(If performed)* All users deleted successfully by the cleanup script.

6. Submission Guidelines

1. Commit **both** scripts (`create_iam_users.py` , `cleanup_iam_users.py`) plus a short `README.md` summarising any challenges.
2. Post the repo link (or zipped folder) in Slack **#general**.
3. Mention one improvement you would add if you had more time (e.g., password rotation with Secrets Manager).

Why this matters

Python is the lingua franca of cloud automation. Re-building last week's Bash workflow in Python teaches:

- virtual-env hygiene → reproducible environments
- `boto3` fundamentals → ready for any AWS service
- error handling with `botocore.exceptions` → robust scripts

- secure secrets management → production-grade practices

Happy coding – and remember to clean up your users when you're done!

LAB | Automating AWS IAM User Creation with Python + boto3

Extra

URL

Enter the URL here

Submit lab